Software Testing and Validation — Project 1

Total: 20 val (Ex1: 6 + Ex2: 6 + Ex3: 6 + Ex4: 2)

Exercise 1: Static Analysis of C (6 val)

In this exercise, you will analyse a third-party C implementation of a Hash table data structure[1] provided in the `ex1` directory. You will use **Infer** and **clang-analyzer** to identify and fix bugs.

1.1 Initial Analysis (2 val)

Run Infer and clang-analyzer on the Hash table program with additional analysis options. For Infer, use the `--pulse` option:

```
infer run --pulse -- clang -c hashmap.c
```

For clang-analyzer, use the additional checkers studied in the lab.

Clearly identify the lines, types of bugs, and whether they are true positives or not in the `template.yml` file provided with the project script. Save this file as `<istid>_ex1.1.yml`.

1.2 Manual Bug Identification (2 val)

Manually identify all bugs in the program. Complete the bug report, saving it as `<istid>_ex1.2.yml`. Provide detailed descriptions of the bugs and specify whether the tools detected them or not. For example:

```yaml
report:
  - bug:
      type: Use-after-free
      lineno: 300
      class: TP   # or FN if not found by any tool
      description: |
        Found with clang-analyzer. (or, Not found by any tool.)
        In function `blah`, if the allocation of the variable `bleh`
        fails, the variable `blih` will be freed and later accessed.
```

1.3 Bug Resolution (2 val)

Create a new version of the file, `hashmap-fixed.c`, fixing all the identified bugs while ensuring that neither Infer nor clang-analyzer produces any meaningful warnings. Add comments with `BUG-x` before each identified bug and `Fixes x` before the respective fixes in the code. Example:

```c
/* BUG-1 */
void *ptr = malloc(length);
/* Fixes 1 */
if (!ptr) exit(1);
```

[1] https://en.wikipedia.org/wiki/Hash_table

Hand-in Instructions

The solutions to 1.1 and 1.2 should be provided in separate files named `<istid>_ex1.1.yml` and `<istid>_ex1.2.yml`, respectively. The solution to 1.3 should be placed in a file named `hashmap-fixed.c`.

Exercise 2: Fuzzing in C with AFL++ (6 val = 1 + 1 + 2 + 1.5 + 0.5)

This exercise relies on the third-party C implementation of a TreeTable data structure[2] provided together with the script of the project. **The same target is reused in Exercise 3**, allowing you to compare the test suites produced by fuzzing and symbolic execution.

2.1 The `balanced()` Oracle (1 val)

Implement a C function `int balanced(TreeTable* t)` that returns `1` if the tree table `t` is *key balanced* and `0` otherwise. In particular, this function should return `1` if for any given node the *height* of the left and right subtrees does not differ by more than 1.

2.2 The `sorted()` Oracle (1 val)

Implement a C function `int sorted(TreeTable* t)` that returns `1` if the tree table `t` is *key sorted* and `0` otherwise. In particular, this function should return `1` when, for any given node, its key is greater than all the keys in the left subtree and smaller than all the keys in its right subtree.

These two functions will be used as **invariant oracles** inside your fuzzing harness: after each operation that modifies the tree, the harness should `assert(balanced(t) && sorted(t))`. This way AFL++ can detect logic bugs that violate the red-black tree invariants in addition to memory-safety bugs.

2.3 Fuzzing Harness (2 val)

Write an AFL++ fuzzing harness for the TreeTable implementation. The harness should:

1. Read a sequence of *commands* from the standard input (use AFL++ persistent mode for performance — see `__AFL_LOOP`).
2. Decode each command as a call to one of the public TreeTable API functions: `treetable_add`, `treetable_get`, `treetable_get_first_key`, and `treetable_get_greater_than`.
3. Insert assertion checks using your `balanced()` and `sorted()` oracles after every state-modifying operation.
4. Properly destroy the TreeTable and free all associated memory at the end of each iteration.

The harness must be designed to **achieve full code coverage** of the four target functions: (1) `treetable_add()`; (2) `treetable_get()`; (3) `treetable_get_first_key()`; and (4) `treetable_get_greater_than()`.

Document the input format used by your harness in a comment at the top of the file. Place the harness in a file named `harness.c` inside the folder `Ex2Harness`.

[2] https://github.com/GillianPlatform/collections-c-for-gillian/blob/master/lib/treetable.c

2.4 Concrete Test Suite (1.5 val)

Compile your harness with AFL++ instrumentation and AddressSanitizer, prepare a small seed corpus that exercises at least one path in each target function, and run AFL++ for a minimum of **30 minutes** on the harness:

```
AFL_USE_ASAN=1 afl-clang-fast -g -O1 harness.c treetable.c -o harness_fuzz
afl-fuzz -i seeds -o outputs -- ./harness_fuzz
```

After fuzzing, minimize the corpus with `afl-cmin` to obtain the smallest set of inputs that preserves the same coverage. Then **convert each minimized input into a concrete C test case** that explicitly replays the corresponding sequence of API calls (i.e. without reading from stdin). Group the tests by the type of property they exercise (validity preservation, lookup correctness, etc.).

Place all concrete tests inside a folder called `Ex2ConcTestSuite`. Each test should be in its own file with a comment at the top describing the property it checks. Crashing inputs (if any) should be saved as separate test files in a sub-folder `Ex2ConcTestSuite/crashes/` and should be annotated with the AddressSanitizer report they produce.

2.5 Coverage Report (0.5 val)

Use the `clang` profiling tool to obtain a *line-oriented* code coverage report for the concrete test suite produced in 2.4. Specifically, build the test suite with `-fprofile-instr-generate -fcoverage-mapping`, run all tests, then merge the profiles with `llvm-profdata` and produce a line-by-line report with `llvm-cov show`. Save the report in a file called `Ex2CodeCoverage.txt`.

Hand-in Instructions

The solutions to 2.1 and 2.2 should be written directly in the file `treetable.c` provided as part of the boilerplate of the project. The fuzzing harness produced as part of 2.3 should be placed inside the folder `Ex2Harness`. The concrete test suite produced as part of 2.4 should be placed inside the folder `Ex2ConcTestSuite`. The coverage report obtained as part of 2.5 should be saved in a file called `Ex2CodeCoverage.txt`.

Exercise 3: Symbolic Testing with KLEE (6 val = 2 + 1.5 + 1.5 + 1)

This exercise targets the **same TreeTable** as Exercise 2. You will produce a second concrete test suite, this time derived from symbolic execution, and compare it with the one obtained through fuzzing.

3.1 Symbolic Test Suite (2 val)

Write a symbolic test suite for the TreeTable implementation. The symbolic test suite should be *property-based*, meaning that each symbolic test should be aimed at checking a specific property of the given tree table implementation (e.g. preservation of validity, correctness of lookup, monotonicity of `get_greater_than`, etc.). Each test should be annotated with a comment describing the property it is meant to check. You should reuse the `balanced()` and `sorted()` oracles from Exercise 2.

The symbolic test suite should guarantee **full code coverage** of the functions: (1) `treetable_add()`; (2) `treetable_get()`; (3) `treetable_get_first_key()`; and (4) `treetable_get_greater_than()`.

3.2 Concrete Test Suite (1.5 val)

Use KLEE to run your symbolic test suite and create a concrete test suite using the concrete inputs generated by KLEE. All concrete tests generated from the same symbolic test should be placed inside the same file, whose name should coincide with that of the corresponding symbolic test.

3.3 Coverage Report (1 val)

Use the `clang` profiling tool to obtain a *line-oriented* code coverage report for the concrete test suite obtained in 3.2. Save the report in a file called `Ex3CodeCoverage.txt`.

3.4 Comparison with Fuzzing (1.5 val)

Write a short report (**maximum 1 page**) called `Ex3Comparison.pdf` that compares the two concrete test suites obtained in Exercises 2 and 3 along the following dimensions:

1. **Coverage achieved** on the four target functions. Did one technique reach lines that the other missed? Why?
2. **Bugs detected (if any)**. Did the two techniques find the same bugs? Did one technique find bugs that the other missed?
3. **Test-generation cost**: time spent fuzzing vs. time spent in symbolic execution; size of each resulting test suite.
4. **Effort to write the harness / symbolic tests**.
5. **One-paragraph conclusion**: in this case study, which technique would you reach for first, and why?

Hand-in Instructions

The symbolic test suite produced as part of 3.1 should be placed inside a folder called `Ex3SymbTestSuite`, with each symbolic test being implemented in its own file. The concrete test suite produced as part of 3.2 should be placed inside a folder called `Ex3ConcTestSuite`. The coverage report obtained as part of 3.3 should be saved in a file called `Ex3CodeCoverage.txt`. The comparison report from 3.4 should be saved as `Ex3Comparison.pdf`.

Exercise 4: Tool Exploration (2 val = 1 + 1)

This exercise asks you to explore **one additional verification or testing tool** of your choice. The tool may belong to the same paradigm as one of the tools used in Exercises 1–3 (i.e., it may be another static analyser, fuzzer, or symbolic-execution engine) — but **we strongly advise picking a fully automatic tool**, as interactive ones (such as deductive-verification systems that require manual proof annotations) typically demand more effort than the exercise's point budget justifies.

Apply the chosen tool to **either** the `hashmap.c` from Exercise 1 **or** the `treetable.c` from Exercises 2–3. Document your work in a single PDF named `Ex4.pdf` containing:

4.1 Tool Description (1 val)

A short text (**maximum 200 words**) explaining: - what category the tool belongs to (static analysis, dynamic analysis, symbolic execution, etc.); - how it relates to and differs from the techniques used in Exercises 1–3; - what kinds of bugs or properties it is best suited to find.

4.2 Tool Application (1 val)

Screenshots of the tool's output on the chosen target program, each followed by a short explanation (a few sentences) of what the output means and how it relates to the bugs in the program. At least three screenshots are expected. Conclude with one sentence stating whether the tool found bugs that Exercises 1–3 missed (or vice versa).

Hand-in Instructions

The solution for Exercise 4 should be given in a single file named `Ex4.pdf`.

Submission Instructions

The project is due on **15th of May**. Be sure to create a zip file containing all the answer files, organised as follows, and upload it in Fenix:

```
<istid>_project1.zip
+-- ex1/
|   +-- <istid>_ex1.1.yml
|   +-- <istid>_ex1.2.yml
|   +-- hashmap-fixed.c
+-- ex2/
|   +-- treetable.c                  # with balanced() and sorted()
|   +-- Ex2Harness/
|   |   +-- harness.c
|   +-- Ex2ConcTestSuite/
|   |   +-- test_*.c
|   |   +-- crashes/
|   +-- Ex2CodeCoverage.txt
+-- ex3/
|   +-- Ex3SymbTestSuite/
|   |   +-- *.c
|   +-- Ex3ConcTestSuite/
|   |   +-- *.c
|   +-- Ex3CodeCoverage.txt
|   +-- Ex3Comparison.pdf
+-- ex4/
    +-- Ex4.pdf
```

Submissions will be closed at 23h59 on the due date. Do not wait until the last few minutes for submitting the project.

Project Discussion

After submission, you may be asked to present your work so as to streamline the assessment of the project as well as to detect potential fraud situations. During this discussion, you may be required to perform small changes to the submitted code.

Fraud Detection and Plagiarism

The submission of the project assumes the commitment of honour that the project was solely executed by the members of the group that are referenced in the files/documents submitted for evaluation. Failure to stand up to this commitment, i.e., the appropriation of work done by other groups or someone else, either voluntarily or involuntarily, will have as consequence the immediate failure of this year's Software Testing and Validation course for all students involved (including those who facilitated the occurrence).